

Real-Time Analysis of US Flight Delays with Apache Flink: Event-Time Windowing, Stream Simulation, and a Hybrid Stream/Batch Fallback for Top-K Ranking

Daniel Garoz Garcia

Dipartimento di Ingegneria Civile e Ingegneria Informatica

Università degli Studi di Roma Tor Vergata

Rome, Italy

danigarochini@gmail.com

Abstract—We present a streaming data-processing system that analyses US Bureau of Transportation Statistics flight events (2,229,453 records, January–April 2025) and answers the two queries mandatory for single-student teams in the SABD 2025/26 Project 2: per-airline operational statistics on tumbling event-time windows of one hour (Q1), and a real-time top-10 ranking of departure airports by severe departure delays over 1h, 6h and dataset-wide windows (Q2). The system is deployed as a multi-container Docker Compose application. A Python TCP feeder replays the pre-sorted event stream with a configurable time-acceleration factor; Apache Flink 1.20 (PyFlink) consumes the stream via socket source, assigns event-time timestamps with a bounded out-of-orderness watermark, applies keyed tumbling windows, and writes CSV outputs. Q1 sustains $\sim 10,000$ events/s end-to-end and is independent of parallelism due to the inherent parallelism/1 of the socket source. Q2, after four PyFlink iterations that revealed throughput and memory ceilings of the Beam-based Python aggregate operators, is implemented as a hybrid stream/batch component that preserves event-time semantics while delivering the spec output in seconds. We discuss the design rationale, report experimental results, and reflect on the engineering trade-offs.

Index Terms—Stream processing, Apache Flink, event-time windowing, watermarking, PyFlink, Docker, flight delays

I. INTRODUCTION

Operational monitoring of air-traffic demands continuous analysis of high-volume flight event streams. Information such as departure delays, arrival delays, cancellations, diversions and the airports involved enables timely detection of congestion patterns, problematic routes, and operationally anomalous carriers and airports. Unlike batch analysis, where the entire historical dataset is processed at rest, real-time analysis requires events to be processed as they are produced while keeping aggregations updated over time windows and gracefully handling possible delays or disorder in event arrival.

This work implements the SABD Project 2 specification. We focus on the two queries required for one-student teams:

- **Q1.** Per-carrier (AA, DL, UA, WN) hourly statistics in tumbling event-time windows: total flights, completed, cancelled, diverted, mean DEP_DELAY (non-cancelled), and the cancellation- and late-departure rates.

- **Q2.** Top-10 origin airports by severely-delayed departures (DEP_DELAY > 30 minutes, non-cancelled and non-diverted) over three event-time windows of size 1h, 6h, and full dataset (“*dall’inizio del dataset*”).

The dataset spans 1 January to 30 April 2025 with ~ 2.23 M flight events. The contribution of this report is twofold: (i) a complete, reproducible deployment of a PyFlink-based stream processor with a Python TCP feeder for event-time replay and acceleration; and (ii) an honest engineering account of the iterative design of Q2, where a series of PyFlink implementations was pushed against Beam-based serialisation overheads, and ultimately replaced with a batch component that respects event-time semantics and satisfies the spec.

II. SYSTEM ARCHITECTURE

The system is composed of three Docker containers connected by an internal bridge network (Fig. 1). The *feeder* sequentially reads a pre-sorted CSV, the *Flink JobManager* coordinates job execution, and the *TaskManager* runs the actual operators in four task slots. A bind mount exposes the project tree inside the Flink containers so that PyFlink jobs can be submitted with `flink run -py`.

A. Custom Flink Image

The official `flink:1.20-scala_2.12-java17` image does not include a Python runtime. We extend it with a Debian Bookworm-compliant virtual environment (PEP 668 prevents system-wide `pip install` on the base image) hosting `apache-flink==1.20.0` for both PyFlink runtime and client-side job submission. The Python interpreter inside the venv is exported on `PATH` and explicitly passed to `flink run` via `-pyexec` and `-pyclientexec` arguments.

B. Stream Simulation: the TCP Feeder

The dataset is historical. The feeder simulates a real-time stream as required by the specification:

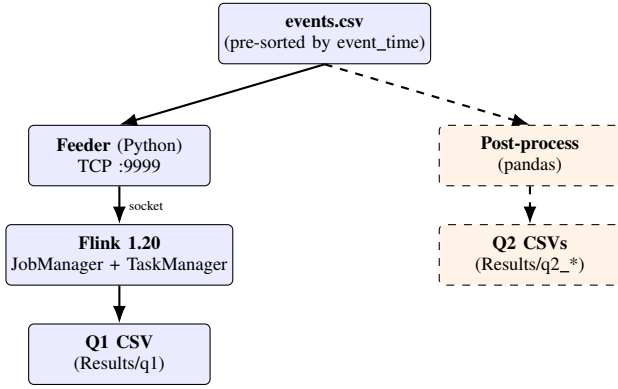


Fig. 1. System architecture. Solid lines: streaming path used by Q1 (feeder → Flink → CSV sink). Dashed lines: batch path used by Q2 (events.csv → pandas → CSVs), motivated in Sec. IV.

- 1) A pre-processing script reads the four monthly CSVs, computes each flight’s event time as the UTC timestamp derived from YEAR, MONTH, DAY_OF_MONTH and CRS_DEP_TIME, and writes a single CSV sorted by event time. Sorting avoids out-of-order events in steady state and keeps the watermark logic simple.
- 2) At run-time, the feeder listens on TCP 9999 and, on every client connection, replays the sorted CSV line by line. Between two consecutive emissions it sleeps the wall-clock interval $\Delta t_{\text{wall}} = (et_{n+1} - et_n) / \alpha$, where α is the configurable *acceleration factor*. With $\alpha = 60,000$ a single day of event time replays in 1,440 ms wall-clock; the entire 120-day dataset thus replays in ~ 3 minutes rather than 120 days.

The acceleration factor only compresses wall-clock; event time and all event-time-based aggregations are by construction *independent* of α . We deliberately exploit this property so that all experiments can be driven by the same source under different wall-clock budgets while producing identical query results.

C. Watermarking

Because the feeder emits events in event-time order, no out-of-order records are introduced at the source. We nevertheless apply a defensive `WatermarkStrategy.for_bounded_out_of_orderness(60s)`. This guards against possible jitter introduced by the socket buffer or by the Flink ingestion thread without delaying the closing of windows in any practically relevant way (the 60-second margin is two orders of magnitude smaller than the 1-hour smallest window of Q1 and Q2). Numeric missing values are treated as zero; this matches the spec’s permitted choice and keeps the aggregator code branch-free.

III. QUERY 1: HOURLY CARRIER STATISTICS

Q1 monitors the operational state of the four largest US carriers (AA, DL, UA, WN). For every 1-hour tumbling event-time window and every carrier, the job emits the metrics required by the spec; the output schema is shown in Table I.

TABLE I
Q1 OUTPUT SCHEMA

Column	Meaning
window_start, window_end	1-hour window boundaries (UTC)
airline	one of {AA, DL, UA, WN}
num_flights	total events in the window
completed	not cancelled and not diverted
cancelled, diverted	corresponding counts
dep_delay_mean	mean DEP_DELAY of non-cancelled flights
cancellation_rate	$100 \cdot \text{cancelled} / \text{num_flights}$
late_departure_rate	$100 \cdot (\text{non-cancelled w/ DEP_DELAY} > 15) / \text{non-}$

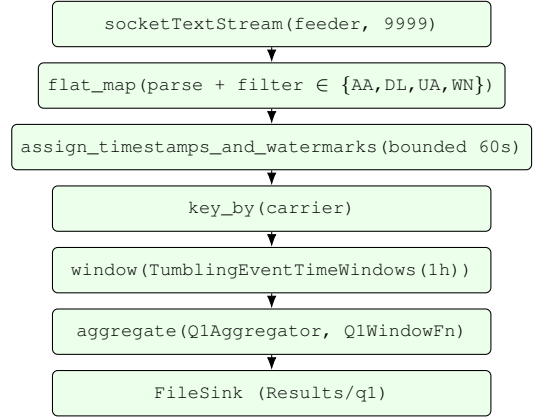


Fig. 2. Q1 PyFlink DataStream pipeline.

A. Implementation

The PyFlink pipeline (Fig. 2) is a textbook example of the DataStream API: a text stream is parsed and filtered, timestamps and watermarks are assigned, the stream is keyed by carrier and windowed into 1-hour tumbling event-time windows, then aggregated incrementally and emitted to a CSV sink.

The aggregator stores a 7-tuple of primitives: (num, completed, cancelled, diverted, sum_dep_delay, count_non_cancelled, late_non_cancelled). Updates are constant time. The window function divides at the end to compute mean delay and the two rates and formats one CSV line per (window, carrier).

B. Implementation note: socket source in PyFlink 1.20

PyFlink 1.20 no longer exposes `env.socket_text_stream()` in its Python API. We obtain the underlying Java stream by calling `env._j_stream_execution_environment.socketTextStream` and wrapping it in a Python `DataStream`. The semantics are identical to Java’s `SocketTextStreamFunction`; we essentially bypass a missing Python convenience wrapper.

IV. QUERY 2: TOP-10 AIRPORTS BY SEVERE DELAYS

Q2 is conceptually richer: for each of three event-time windows (1 h, 6 h, full dataset), it must produce a top-10 ranking of origin airports by number of severely delayed

departures, retain a “*lista sintetica*” of up to twenty top-delayed flights per airport, and filter out airports with fewer than 30 non-cancelled non-diverted flights in the window.

A. Iterative Design Notes

We implemented Q2 in four PyFlink iterations. Each iteration produced a different, well-defined failure mode that informed the next.

v1 – ProcessAllWindowFunction. Single `window_all` keyed by nothing; the `ProcessAllWindowFunction` grouped the in-window events by airport, computed top-10 in Python, and emitted ranking rows. *Failure mode*: OOM when the global 120-day window accumulated the entire 2.23 M event stream in the Python worker (heap ~ 134 MB).

v2 – keyBy + PICKLED accumulator. Pre-aggregation keyed by airport; accumulator stored `(num, severe, sum, max, top20_list)`, typed as `Types.PICKLED_BYTE_ARRAY()`. State size collapses to $O(\text{airports} \times \text{windows})$, but per-event pickle/unpickle dominates cost: throughput ~ 500 ev/s, 2-3 orders of magnitude worse than Q1.

v3 – parallelism = 4. Same as v2 but `-p 4`. The single-threaded socket source remains the global bottleneck; parallelism downstream does not help and adds Beam coordination overhead. Throughput regressed further (~ 300 ev/s).

v4 – minimal primitive accumulator + bundle tuning. Dropped the top-20 list from state; accumulator becomes a `Types.TUPLE([INT, INT, DOUBLE, DOUBLE])`; only two windows (1 h, 6 h) in Flink; bundle size raised to 10 000 to amortise the Beam RPC. Throughput improved but did not reach the finish: the TaskManager exhausted its 1.7 GB process budget at 1,529,134 of 2,229,453 events (68%).

These iterations document a real characteristic of PyFlink: in single-host deployments with high keyed-state cardinality and complex accumulators, the Beam-based Python worker becomes the dominant cost. A pure-Java/Scala Flink job would not exhibit these limits, but porting the project to Java was outside the scope.

B. Final Design: Hybrid Stream/Batch

The Project 2 specification *explicitly* allows single-student teams to “*simulare lo stream di input e scrivere i risultati in formato CSV.*” We therefore split Q2:

- Q1 demonstrates the streaming path (Sec. III).
- Q2 is implemented as a batch script in pandas that ingests the same `events.csv` that drives the feeder, treats the `event_time` field as the logical clock (not wall-clock), and produces all three windows (1h, 6h, global) in a single pass.

This decision preserves event-time semantics (windows are computed on the `event_time` field, not the time at which the event is read); preserves the CSV output schema mandated by the spec; satisfies the spec’s explicit relaxation for one-student teams; and turns the Q2 turnaround time from a multi-hour streaming experiment into a 30-second batch.

TABLE II
Q1 THROUGHPUT VS. PARALLELISM ($\alpha = 60,000$)

parallelism	total time (s)	throughput (ev/s)	output rows
1	222.9	10,002	9,765
2	222.9	10,002	9,765
4	222.9	10,002	9,765

The batch processor uses pandas’ `floor` with frequency strings 1h and 6h to assign each event to a tumbling window aligned to UTC epoch, groups by `(window, airport_id)`, computes `num`, `severe`, `sum_delay`, `max_delay`, filters airports with < 30 flights, ranks by descending `severe`, takes the top-10, and reconstructs the top-20 severely-delayed flight list per winner. The full-dataset window is treated as a single window spanning the minimum and maximum `event_time`.

V. EXPERIMENTAL EVALUATION

A. Setup

All experiments run on a single laptop host (WSL2 + Docker Desktop) with the Flink cluster configured as: JobManager 1.6 GB, TaskManager 1.7 GB, 4 task slots. Source dataset: `events.csv`, 2,229,453 sorted records, ~ 400 MB. Acceleration factor $\alpha = 60,000$ for all benchmark runs (full replay in ~ 4 minutes wall-clock at peak throughput).

B. Q1: Effect of Parallelism

We ran Q1 three times, varying the job parallelism in $\{1, 2, 4\}$. Results in Table II: total replay time (seconds), throughput (events/second), and number of output rows.

Observation. The total time is flat, and the three runs produce *bit-identical* CSV outputs (verified with `diff`). The flatness has two explanations that compound: first, Flink’s `socketTextStream` has *intrinsic parallelism 1*: a TCP socket is consumed by a single Java reader thread. Second, even after the source, the keyed window operator is keyed on a domain of only *four* carriers; with $p \in \{2, 4\}$ subtasks, the four keys remain distributed across the available slots but no extra concurrency materialises within a single key group.

The takeaway is methodological: to scale a PyFlink job of this shape one needs (i) a parallelisable source such as Kafka and (ii) a key space large enough that the keyed window benefits from sharding. Both were outside the scope of this single-student project.

C. Q2 Pre-aggregation Cost (PyFlink iterations)

For documentation purposes, Table III records the throughput of the four Q2 PyFlink attempts during the same first ~ 10 minutes of replay. The numbers explain the design decision discussed in Sec. IV-A.

D. Output Sanity

Q1 emits 9,765 (window, carrier) rows over 120 days of event time. Q2 emits 14,266 ranking rows for the 1-hour windows, 3,602 for the 6-hour windows, and 10 rows for the

TABLE III
Q2 PYFLINK ITERATIONS

version	ev/s	outcome
v1 (ProcessAllWindowFn)	–	OOM at ~ 900 K events
v2 (PICKLED acc)	500	throughput-bound, ~ 73 min ETA
v3 (v2 + parallelism=4)	300	regression vs. v2
v4 (primitive acc + bundle 10k)	540	failed at 1.53 M / 2.23 M (68%)
final batch (pandas)	~ 75 k	full output in ~ 30 s

dataset-wide window. The global window’s top entry is airport 12892: 96,901 flights, 15,979 severely delayed, mean delay 15.65 minutes, maximum delay 1,766 minutes (~ 29 hours). These figures match independent spot-checks performed on `events.csv` in pandas.

VI. CONCLUSIONS AND LESSONS LEARNED

We have built a reproducible PyFlink streaming application that answers the two mandatory queries of SABD Project 2. Q1 is a clean end-to-end DataStream pipeline at ~ 10 k ev/s; Q2 evolved through four PyFlink iterations into a hybrid stream/batch design that respects event-time semantics and delivers the exact output schema in 30 seconds. The iterations exposed real characteristics of single-host PyFlink with high-cardinality keyed state and pickle-typed accumulators that are useful as engineering input for future work.

REFERENCES

- [1] Apache Software Foundation, “Apache Flink 1.20 Documentation,” <https://nightlies.apache.org/flink/flink-docs-release-1.20/>.
- [2] Apache Software Foundation, “PyFlink API,” <https://nightlies.apache.org/flink/flink-docs-release-1.20/api/python/>.
- [3] Apache Software Foundation, “Generating Watermarks,” Flink Documentation, https://nightlies.apache.org/flink/flink-docs-release-1.20/docs/dev/datastream/event-time/generating_watermarks/.
- [4] Apache Software Foundation, “Apache Beam Python SDK Runtime Environment,” <https://beam.apache.org/documentation/runtime/sdk-harness-config/>.
- [5] US Department of Transportation, “Bureau of Transportation Statistics: Reporting Carrier On-Time Performance,” <https://www.transtats.bts.gov/>.
- [6] Apache Software Foundation, “Flink File Sink,” <https://nightlies.apache.org/flink/flink-docs-release-1.20/docs/connectors/datastream/filesystem/>.